

AJX学习与感想

AJX架构介绍

在小b商家项目中算是我们第一次接触AJX框架，经过一个大项目，一直到现在对他的感受可谓又是爱又是恨。项目中我们只接触了AJX的主要部分-动态化UI。对其整体的架构缺乏一个全面的认识。

AJX是一套基于JS引擎、用于实现移动端跨平台应用开发的系统级解决方案，分为AJX前端、AJX引擎层和IOS/Android适配层。其中AJX前端是我们平时接触最多应用最多的，虽然我们不熟悉引擎层和适配层，但是他们三者关系紧密，缺一不可。

AJX的由来

随着高德业务发展越来越迅速，需求更新迭代快，如果采用最原始的方案分别由IOS和Andriod去开发，人工成本大，需要双方都开发一套代码，而且IOS需要经过苹果的审核，上线周期慢。

然后是前几年呼声很高的Hybird App，前端只需要按照html5+css3+js开发即可，然后打包生成可以支持IOS和Andriod的类似原生的应用，运行在app内置的浏览器上。属于半原生半Web，缺点也很明显，性能不足，一些复杂的动画或像地图导航这类交互性强的也无法得到很好的实现，同时前端需要熟悉原生，学习成本较大。

再之后是Facebook推出的RN以及淘系的开源框架Weex，他们的效率和体验接近Native App，发布和开发成本低于Native App，但RN涉及底层的功能需要Android和Ios双端单独开发，由JS调用，这点和Hybird App类似，这对于native开发者更加友好，对于前端来说还是需要学习熟悉原生，开发成本较大，而且React Native 严重依赖于 Facebook 的维护，开发中不免有些疑问，只能在社区查找答案或者等待官方回复，效率慢。说到这里，相信很多人还记得2017年Facebook曾发生过一个专利条款事件，条款大致是说，只要你在用React，Facebook侵权你的专利你也不能告他，而且不只是跟前端相关的专利，包括了你所拥有的所有专利，否则在你提起诉讼的瞬间，你就失去了使用React的授权。当时带头的百度以及Apache等公司都要求停止使用React / React Native 等 Facebook 下涉及特定专利条款的开源产品，当时整个互联网圈因为这个事弄得沸沸扬扬，但结果是好的，最终成功逼迫Facebook更改了条款内容。而AJX也正是在这个时候诞生的，没有选择RN不知道是不是也有这部分原因。

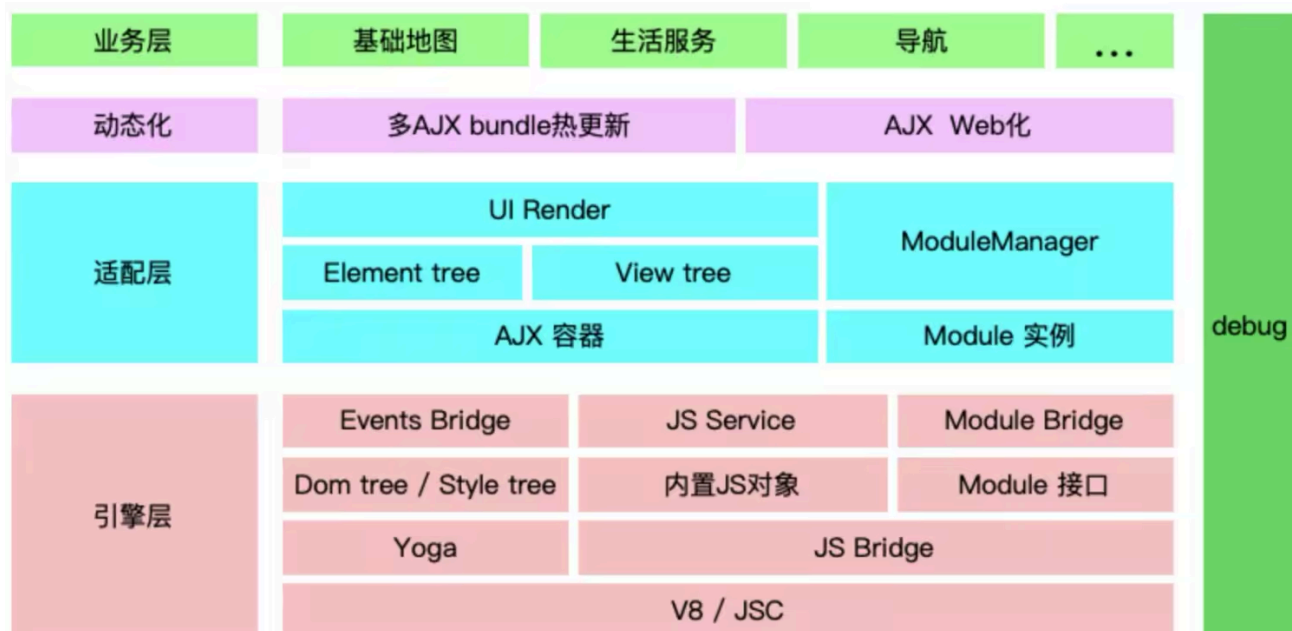
关于淘系的Weex框架，诞生初期确实呼声很高，但是之后也因技术瓶颈一直停滞不前。技术方案比较复杂，双端差异性一直无法很好地统一，开发体验也不是很好，和RN相比也有一定差距，无法很好地解决高德一些业务逻辑。目前Weex2.0正在开发落地中，这次的架构升级基于Flutter，整个架构正朝着更好的方向发展。

在这样的一个背景下，我们选择了自己去研发一套符合地图，导航复杂交互业务场景的框架，AJX也由此诞生。目前我们使用的AJX 3.0 在框架重构时借鉴了当时比较火热的react native

的部分框架设计，布局引擎切换成了facebook开源项目Yoga。相信如果是现在再去重构选型，一定会有更好的优秀框架可供选择，比如Flutter。

AJX框架设计

AJX的框架设计可以细分为4层：



业务层

业务层包括各个业务线，各个业务线之间没有依赖，就像我们b端和c端，属于两个不同的项目，打包之后也是两个不同的bundle，他们可以共同引入公共bundle，使用公共bundle提供的一些公共组件或资源。其中业务层的实现靠的是前端UI层，也就是我们日常开发的部分，一般都基于前端框架FOX进行开发，AJX提供的都是原生api，包括xml书写、dom操作、渲染、事件处理，FOX将这些api进行封装，使UI层的开发能达到类似在浏览器上的开发体验。

AJX的UI层发展历程

○AJXFramework（已废弃）

AJXFramework作为AJX最初的框架，在一个时间紧急，业务开发和基建需要同时进行并上线的情况下诞生。为了不阻塞业务开发，只能摒弃接入主流框架，将JSX语法实现基本组件化，也具有了生命周期的定义。并为高德落地了一些较为简单的业务。但是AJXFramework只是一个UI层解决方案,虽然也实现了部分生命周期，虽然在语法采用JSX，但AJXFramework本身没有脱离dom操作的模式（通过给组件绑定ref，用this.refs去操作dom）。这更像是个react-like。随着业务越来越多，AJXFramework根本无法满足越来越复杂的业务，寻求新的框架迫在眉睫。

○Fox

新的框架并没有选择React或Vue，同时为了避免重复造轮子，选择了淘系开源框架Rax，配合定制化dva形成了Fox框架。

其中Rax框架是用于构建移动跨端应用的渐进式 React 框架，并致力于解决跨容器渲染，实现了 80%+ 的React API，大部分react生态（rax-redux、dva、mobx等）也可以无缝接入。相比React，它轻量很多，非常适合无线端。

而 dva 是基于现有应用架构 (redux + react-router + redux-saga 等)的一层轻量封装，全部代码不到 100 行。dva没有引入任何新概念，并很明确地告诉你每个部件应该怎么写，这对于团队而言，会更可控。至于定制化dva，是由于redux-saga在AMap下的IO和执行时间在Iphone 5s上几乎要花掉80ms，在保持dva核心语法的前提下解除了dva和redux-saga的绑定，替换成了我们自己开发的一个轻量的异步中间件。

动态化

动态化分为热更新能力和Web化能力。

热更新能力：主要用于修复线上业务bug或Crash，通过动态下载bundle文件的增量包并热加载得以实现；需要热更新的bundle必须在App内有对应的随版bundle.

Web化：主要用于解决新业务在线上App上线的问题，即一个没有随版的bundle可以通过发布服务，在线上App上线并生效

JS引擎层

JS引擎层是链接JS与适配层之间的桥梁，包含了JavaScriptCore/v8库。其中JavaScriptCore和V8分别是苹果和谷歌的开源JavaScript引擎，作用于IOS和Android系统上，由于V8引擎在初始化时间上和内存占用上都不如JavaScriptCore，引擎层经过尝试、重构，Android已经在最新版本全面使用JavaScriptCore引擎，并稳定运行中。JS引擎层主要作用是解析JS和xml，生成DOM树，布局计算并通知适配层刷新UI。

JS适配层

适配层包含IOS和Android，可以将引擎层解析后的DOM树用原生组件渲染出来。

渲染能力

UI层绑定的事件对页面的操作都会被引擎层捕获，然后转化为事件，并把相应的样式、属性等封装在事件对象内，再传递给适配层，适配层根据不同事件类型，对应到不同原生组件的渲染和样式以及属性的修改上。

AJX-iOS适配层

根据不同事件进行原生组件渲染

事件及数据回调

AJX引擎层

针对节点进行事件及数据封装

AJX业务层

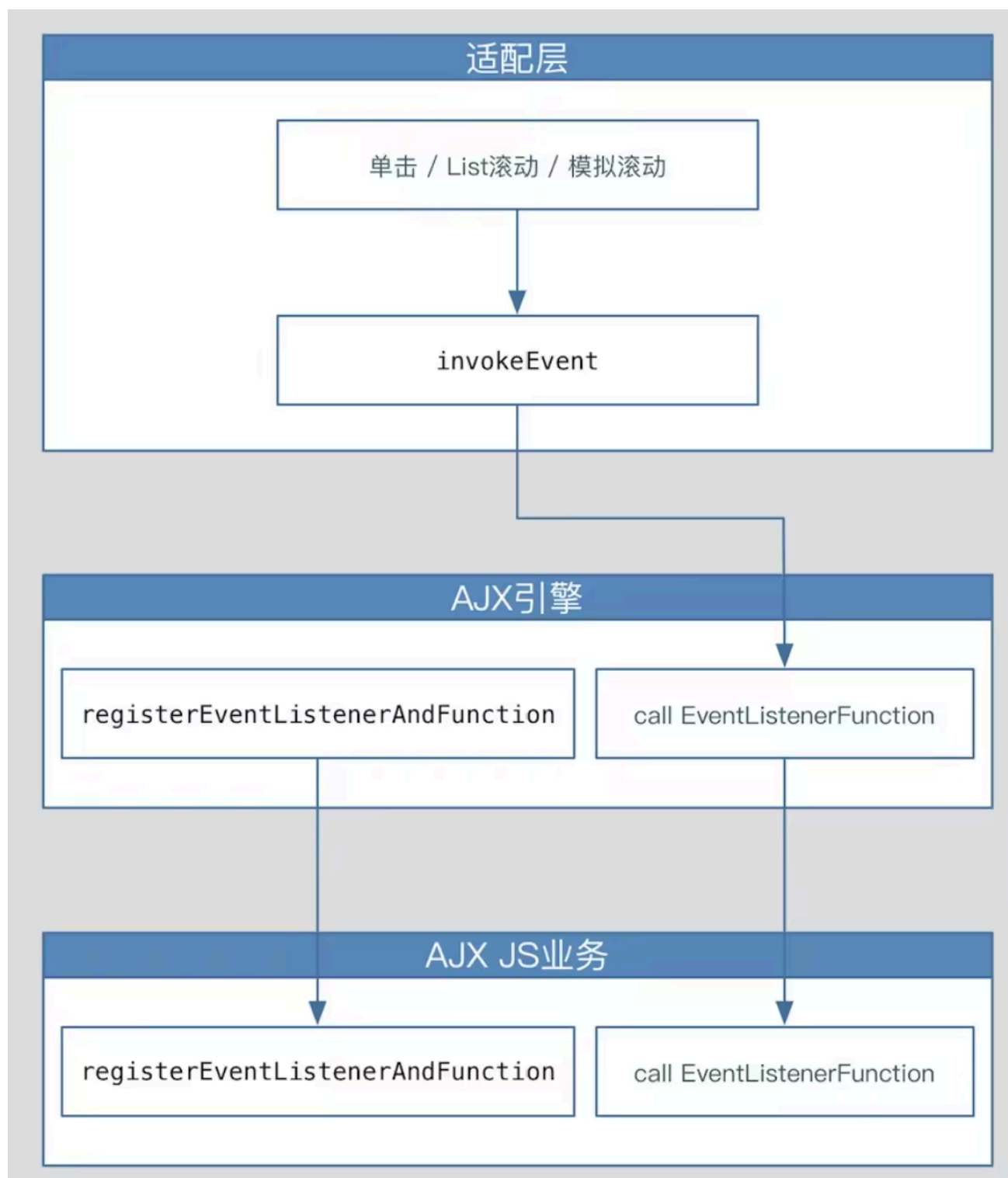
XML解析

节点增、删

节点属性增、删、改

交互能力

页面上所有交互事件，均由适配层触发，通过AJX引擎传递到AJX JS业务中



AJX的热更新

热更新是AJX的核心功能之一，业务能够不依赖APP包版本发布限制，较为自由，及时的发布新功能，修复已有bug。这个对于IOS来说应该是最有优势的，省去了IOS漫长繁琐的审核。为什么说较为自由呢，据我所知，苹果从一开始就对所有上架的APP有着严格的审核，并且审核也在逐渐收紧。2017年，发生了一件关于苹果热更新的事件，很多IOS开发者收到了苹果的警告邮件，禁止了部分热更新的函数、框架和代码，但未禁用热更新机制。很多人都猜测像RN，Weex这类都会受影响，事实证明RN，Weex并没有受到影响，像可以直接修改ios方法的JSpatch倒是被禁了，只不过因为它违背了苹果对应用生态的安全可控和体验一致性的要求。但是苹果的审核确实越来越严了，有机审和不定期的入审抽查，我的上家公司就遇到过这类问题。经过咨询AJX框架的同事，目前我们AJX需求都是不随版，直接作为patch单独上线，目前也没有遇到审核不通过的情况。这样热更的优势就完全体现出来了。

ajx的热更流程：

冷启动

○冷启动

- i.如果是App升级后的冷启动，首先删除所有的ajx patch文件。
 - ii.如果是非App升级后的冷启动，首先删除，除最新一次下载的ajx patch文件之外的patch文件。
- 初始化AJXFileLoader，加载随版ajx包base.ajx和本地最新的patch包patch_i.ajx。
 - 检查是否有新的patch包(patch_i+1.ajx)需要下载。
 - 下载最新的patch包(patch_i+1.ajx)到更新目录。
 - 校验更新包(patch_i+1.ajx)是否合法并添加到AJXFileLoader。
 - AJXFileLoader建立cookie_n与patch_n的对应关系。

热启动

- App从进入后台再回到前台的时间大于1小时的时候，检查是否有新的patch包(patch_i+1.ajx)需要下载。

AJX的前端生态

AJX对于前端来说，它的UI框架，IDE，调试，物料平台，日志采集，项目拆分bundle化、Lowcode都是比较符合前端生态的。但是AJX并没有完全采用W3C标准。举个例子，出于性能考虑，AJX并没有采用像ul、li这类WEB标准中的标签，而是采用了AJX特有的标签进行列表渲染。诸如此类的很多差异使AJX生态与集团现有WEB技术体系生态不完全兼容。

FOX UI框架

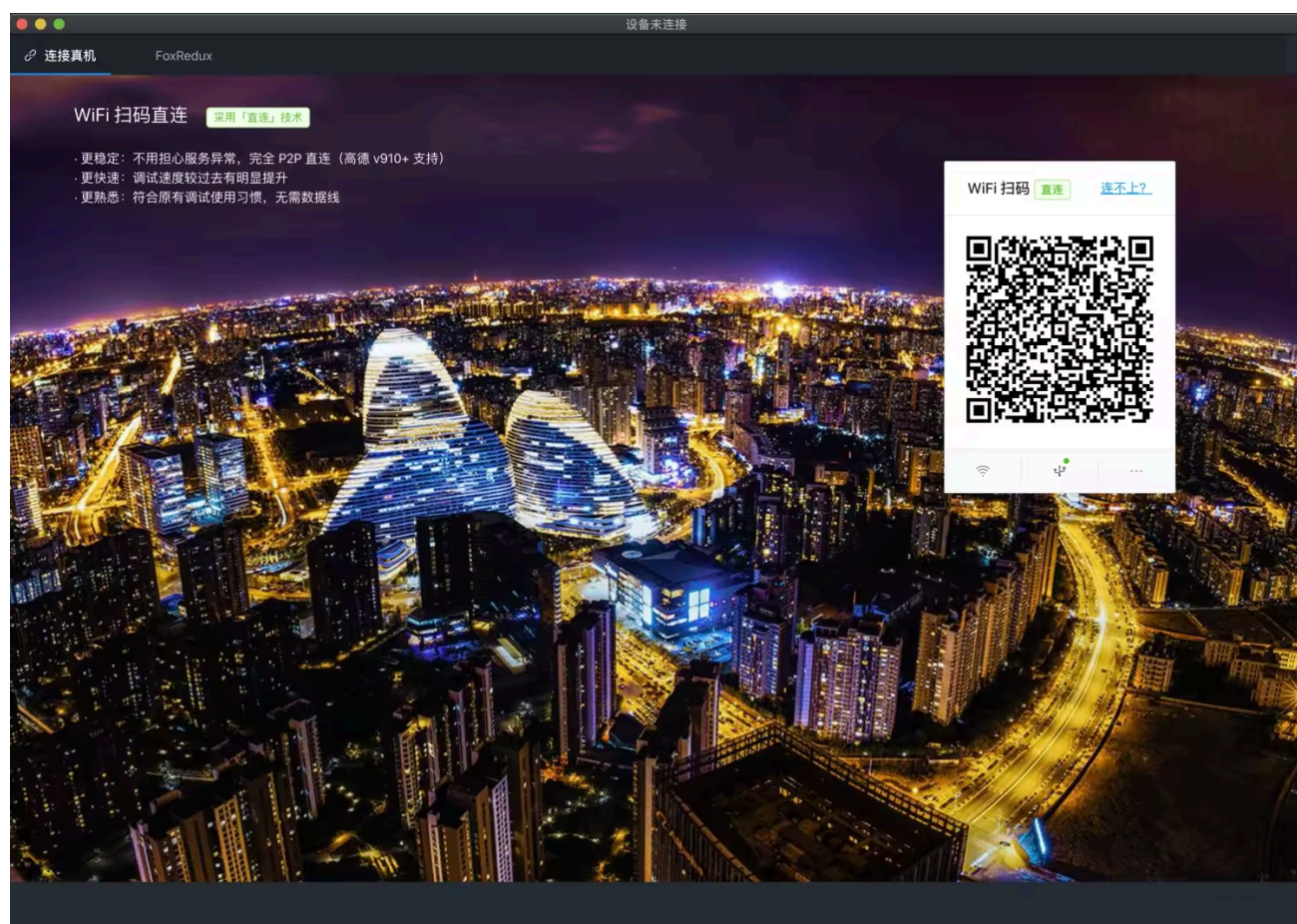
FOX的写法类似React+Redux，并且支持less、es6、npm、ts。就开发而言，对于native同学和前端同学来说都很容易上手。

IDE

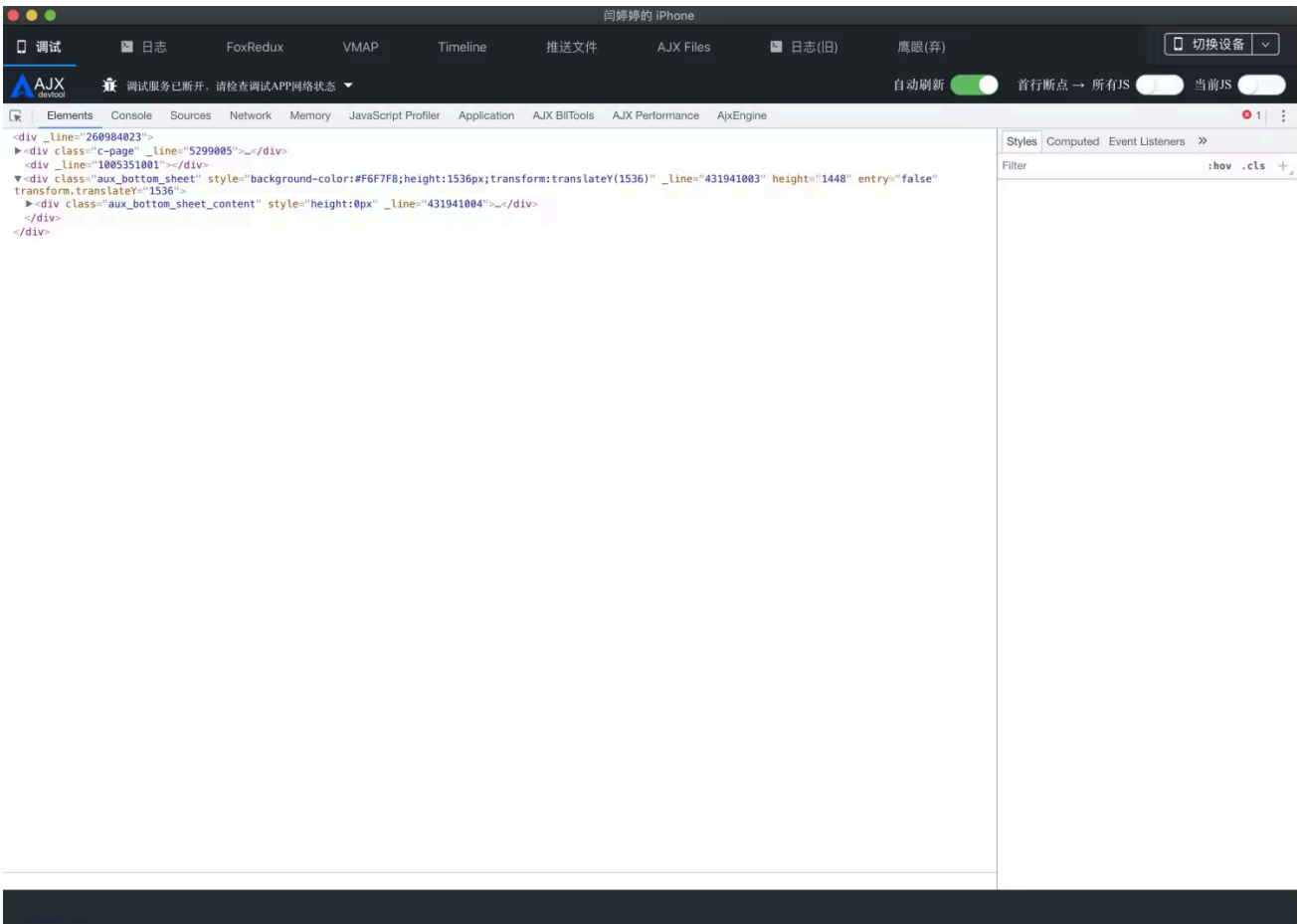
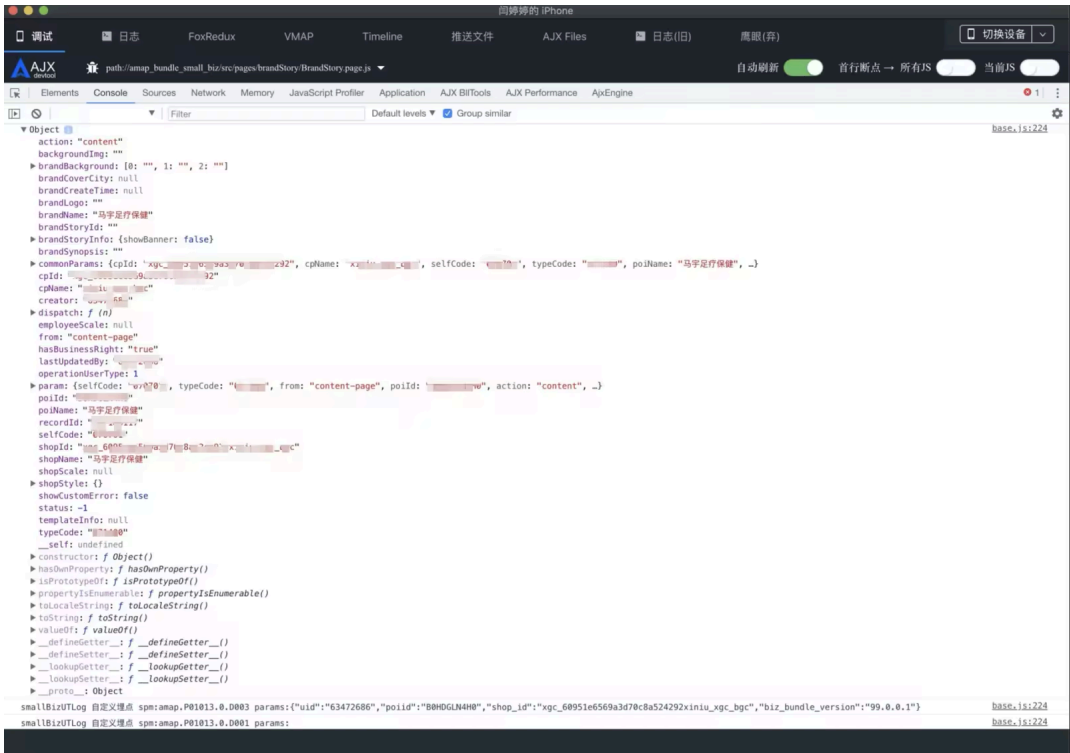
AJX定制的IDE集成了编译，调试等定制化功能，整体上还是类似市面上主流的IDE。IDE默认使用VS code的配置和快捷键，使用起来也和VS code几乎一样，更加贴心的是对于习惯使用WebStorm、Sublime、Atom、XCode、Vim的小伙伴提供了对应配置教程。

调试

调试工具仅仅需要用AMap的调试扫码工具扫一扫即可链接真机进行调试，无需数据线，方便快捷，另外性能分析也是AJX一大亮点。



其中调试部分和chrome浏览器看到的调试控制台十分类似，可以审查元素，查看输出的log，以及断点调试，查看网络接口请求。



物料平台和业务 bundle 化

物料平台集成了AJX的一些公共组件和方法，在项目中的ajx_modules中，在项目开发的页面可以引入并使用。而且物料平台打通了设计开发一体化，设计在done上输出设计图时可以标注具体哪

部分使用哪个公共组件，点击可以直接跳转到对应Acorn组件文档页，使前端开发便捷。

AJX不同业务为不同bundle。公共组件，公共库在项目中通过依赖引入。这和前端开发的流程没有区别。但是Ajx的不同bundle间可能存在相互跳转，对于web和h5，不同业务间可以直接通过url跳转，根域相同共享登陆信息。Ajx采用了 protocol 的引用方式，既明确了 bundle 之间的依赖关系，又保证了 bundle 对外输出的稳定性，同时由于 bundle 内只有显示静态声明可被外部使用的功能才能被外部使用，成功限制了隐性的依赖，示例

```
{
  "protocol": {
    // 向外暴露的功能
    "router": {
      "scheme": {
        // key 为对外提供的功能名称, value 为提供此功能的文件
        "amapscheme://search/homepage": "homepage.js"
      },
      "service": {
        // key 为对外提供的功能名称, value 为提供此功能的文件
        "amapservice://search/searchDistance": "search_distance.js"
      },
      // 其它功能
    },
    // 依赖的外部业务功能
    "dependencies": {
      "service": ["amapservice://user/login"],
    }
  }
}
```

Lowcode

又一个提高前端开发效率的好平台，Lowcode搭建平台接入了高德设计研发一体化工程团队的工作成果，拥有了设计稿一键转搭建代码的基于集团ImgCook的D2C能力，开发人员拿到Done设计稿后，可以选择一键跳转到Lowcode搭建平台。进入平台后，样式布局自动生成，直接省掉大部分设计稿还原时间，最终，经过数据编排，FaaS投放等环节，就是我们在高德地图首屏上滑Panel后看到的场景推荐卡片。虽然这个平台目前还有很多缺陷，只能针对相对简单，没有交互的展示页面，但是这一定是未来的趋势，相信之后Lowcode会越来越完善，不仅是开发可用，产品、运营也可操作快速生成一些活动宣传页。

日志平台

日志平台一般对于前端排查线上问题来说都是必不可少的。毕竟用户的一些操作方式，网络环境等我们都是无法知道的，如果出现了问题，一般的用户都是弃用或换其他更好的竞品。所以AJX也接入了ALC全链路日志，方便在代码中增加埋点，及时上报问题。以及Hubble监控报警平台还能及时将触发警报的问题日志发送到客户端钉群并@相关负责人。这样完善度日志机制，使开发人员发现问题，定位问题，解决问题更加高效，有利于我们整个公司的口碑和稳定发展。

AJX的一些吐槽

前面夸了那么多，接下来也该说说AJX一些让人开发难受的地方，虽然有AJX文档，但是很多开发中的问题文档并没有涉及，还得亲自去踩坑。

代码开发

print输出log

对于习惯在浏览器开发的同学来说，一开始以为print的实现和console.log是一致的，所以我在最开始出现了这样的写法,发现log是打出来了，但是为什么没有值呢。后来才了解到原来print不支持这种写法，只有一个参数。

```
print('props',this.props)
```

CSS

通过AJX文档就可以发现，AJX的css是想要尽量靠拢W3C的，但是无奈兼容两端设备确实还存在很多鸿沟，所以目前只能接受，据文档统计，AJX只支持W3C中一小部分，不过这一小部分也是我们最常用的，基本可以满足日常开发。

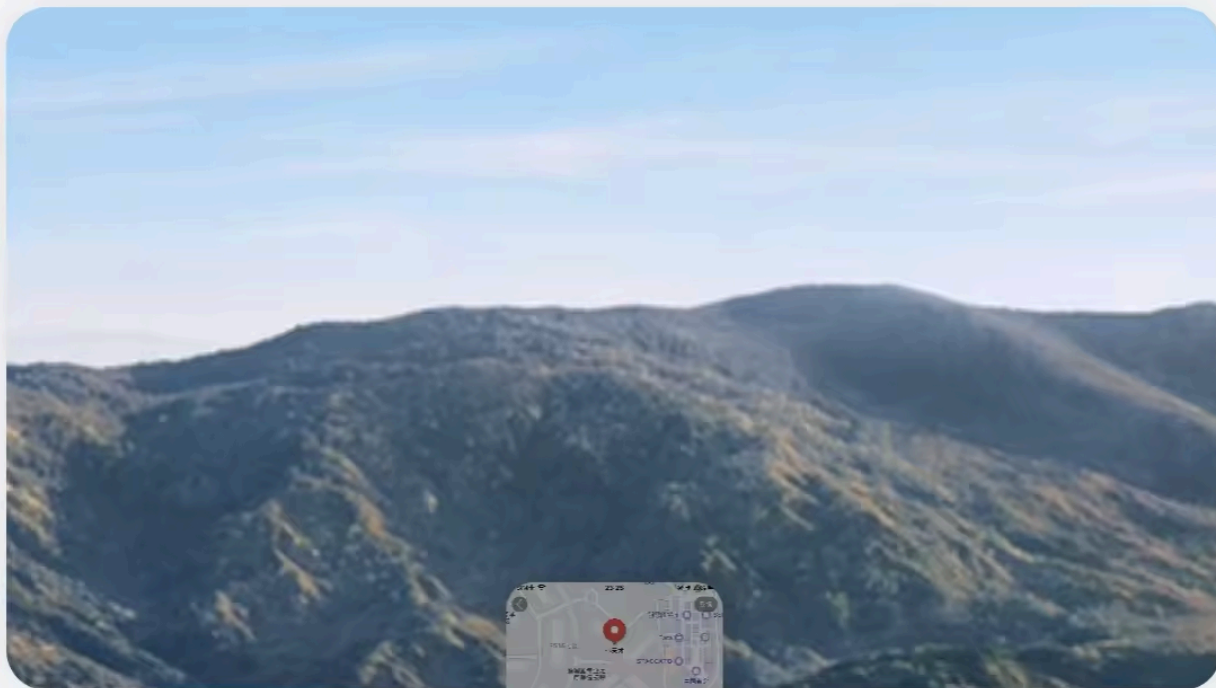
举几个作为前端日常经常用到的样式，flex:1，border-bottom: 1px solid #ddd; background-color: transparent; z-index: 1; 以上这些样式通通不支持，其中flex:1是不支持这种简写方式。

关于组合选择器，习惯用less的同学都习惯子孙元素嵌套在父元素css下的写法，但是AJX只支持两级，仅支持父->子，而嵌套子元素的子元素css就失效了。

组合选择器

W3C	AJX
相邻兄弟选择器 A + B	不支持
普通兄弟选择器 A ~ B	不支持
子选择器 A > B	不支持
后代选择器 A B	功能相当于子选择器

最近刚踩了overflow的坑，如果父元素设置了visible，子元素有超出部分，超出部门是无法响应点击事件的。需要在父子元素外包一层，保证父子都在该容器内。



艺龙足疗店

 黑框区域手势子控件无法响应

父 overflow:visible

子



IDE开发工具

命名

初次使用AJX的小伙伴一定有发现，在定义文件名时，如果已经有文件夹内部的page或less文件名字为index.page和index.less，而新增的文件夹内部文件如果仍定义index.page，index.less，IDE会报错提示文件重名。之前AJX互帮群也有同事提及过此问题，觉得这不符合前端正常的开发思维，建议负责IDE的同事能支持paths 功能，这样就可以通过配置路径映射的方式，使用相对目录导入模块。但是结果不尽如人意，工具组同事给出了解释，改动成本大，收益不大。不过就这点来说其实影响也不大，适应这种模式就好了。



叶俊星(叶俊星) 🐾 封闭项目中，工具链问题请联系 ...

这个功能如果要支持，需要编译器、IDE、笛卡尔（依赖分析）等一系列周边配套设施同时支持，成本上来说比较高。从收益的方式来说，目前看是仅仅能文件命名时可以用一样的，而这我判断不是必需的，而且是对当前「可以只写文件名」的规则形成冲突。整体上来看收益是负的。

我的观点是，规范并没有绝对的正确，大部分都是主观感受，不影响业务效率和质量的情况下，有个标准就可以了。



翟旭光(翟旭光)

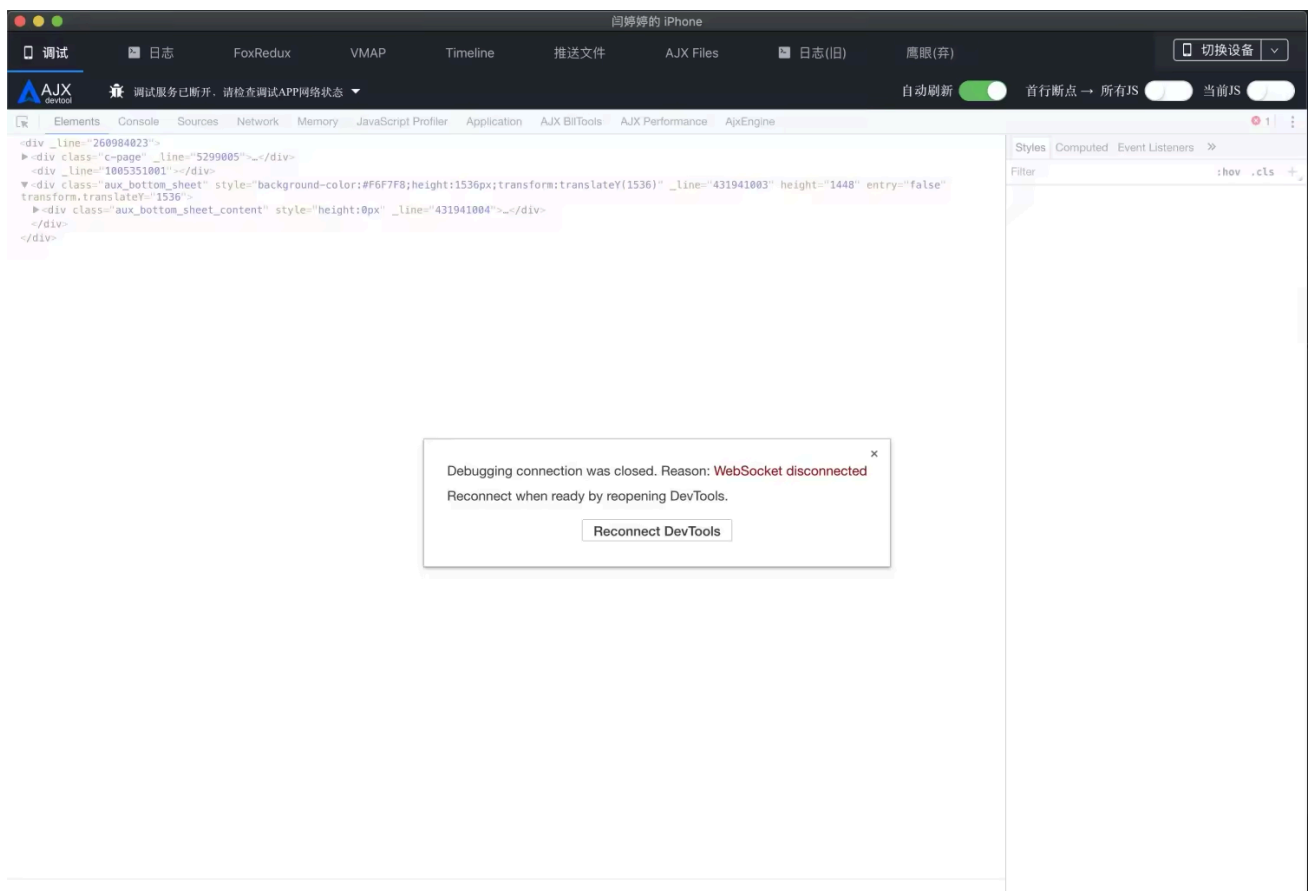
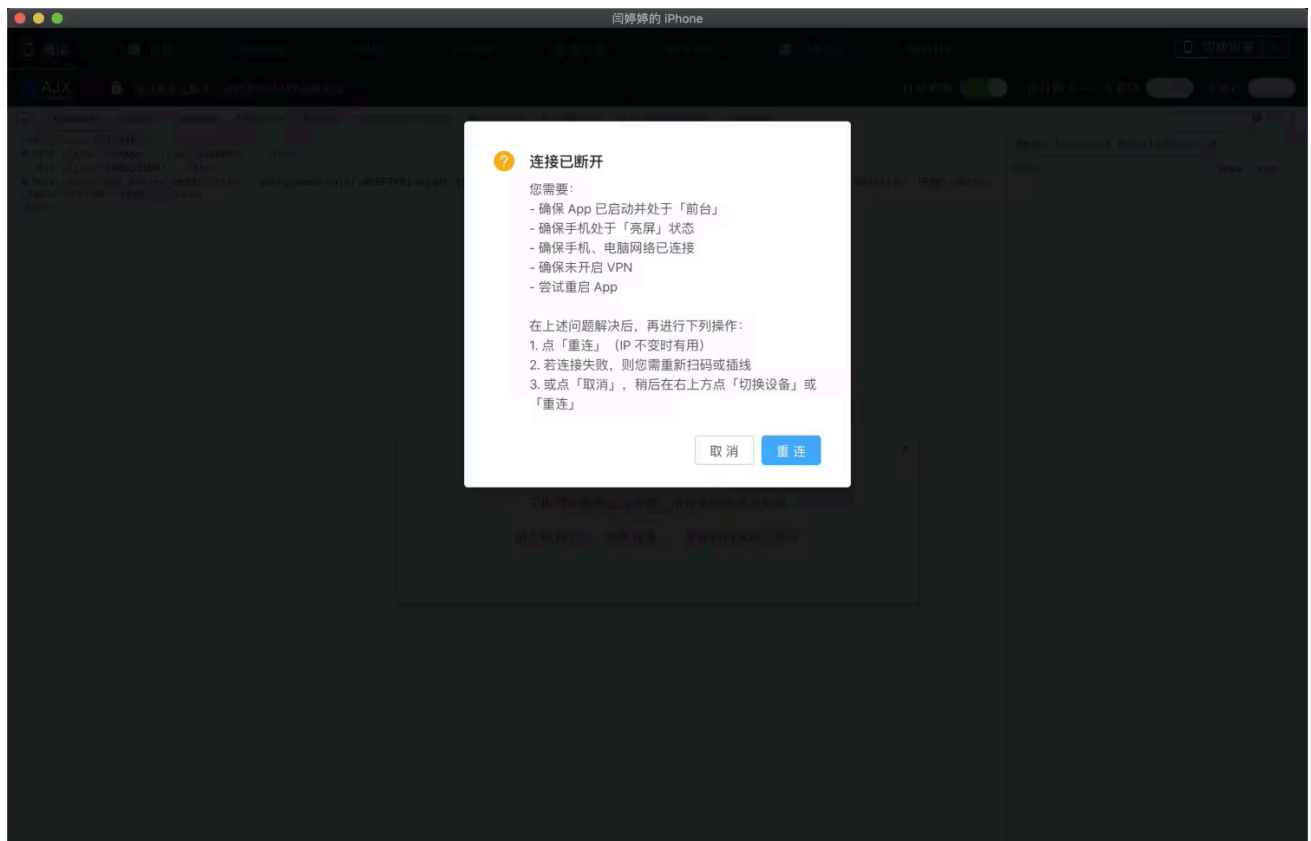
webpack的resolve.alias和typescript的paths都是编译时做替换，不影响打包产物。不需要运行时环境做改动，我们也可以实现，但是成本比较高，roi比较低。

编辑慢

随着项目文件越来越多，编译速度也越来越慢，而且没有进度条显示进度，不知还要等多久。据说工具组同事已经把这个提上日程，后续会增加进度展示。

调试工具

○如果APP有一段时间没有在前台，调试工具需要重启，比如下面的情况必然会断开，这时候点击重连，点击Reconnect DevTools也没有用了，只能重新扫码。



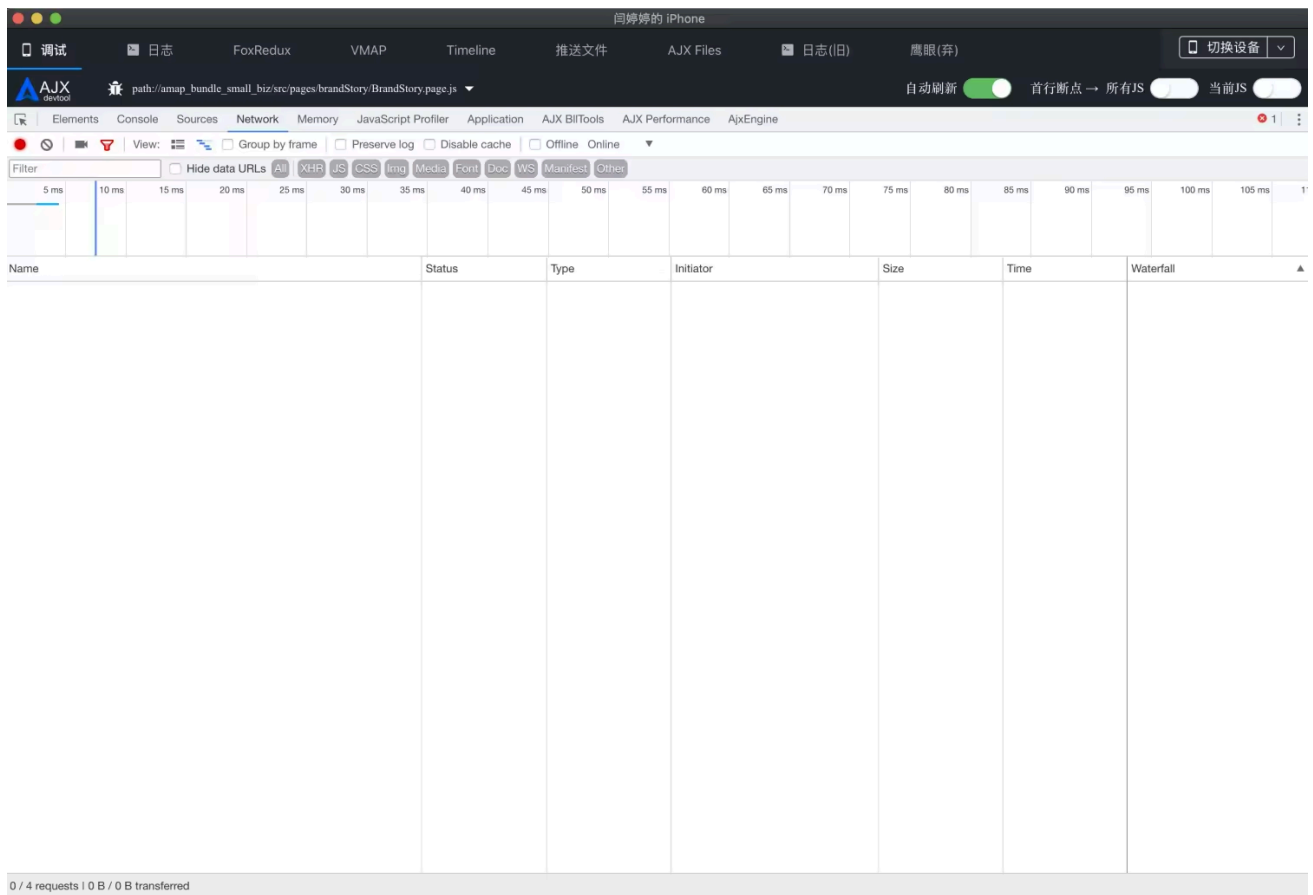
○扫二维码展示不出来，重新开启调试工具还是不行，重启IDE倒是可以了。咨询了工具组同事，原来空白一般就是由于代理软件对网络进行代理而导致的，需要在代理软件上配置一下 localhost 白名单。



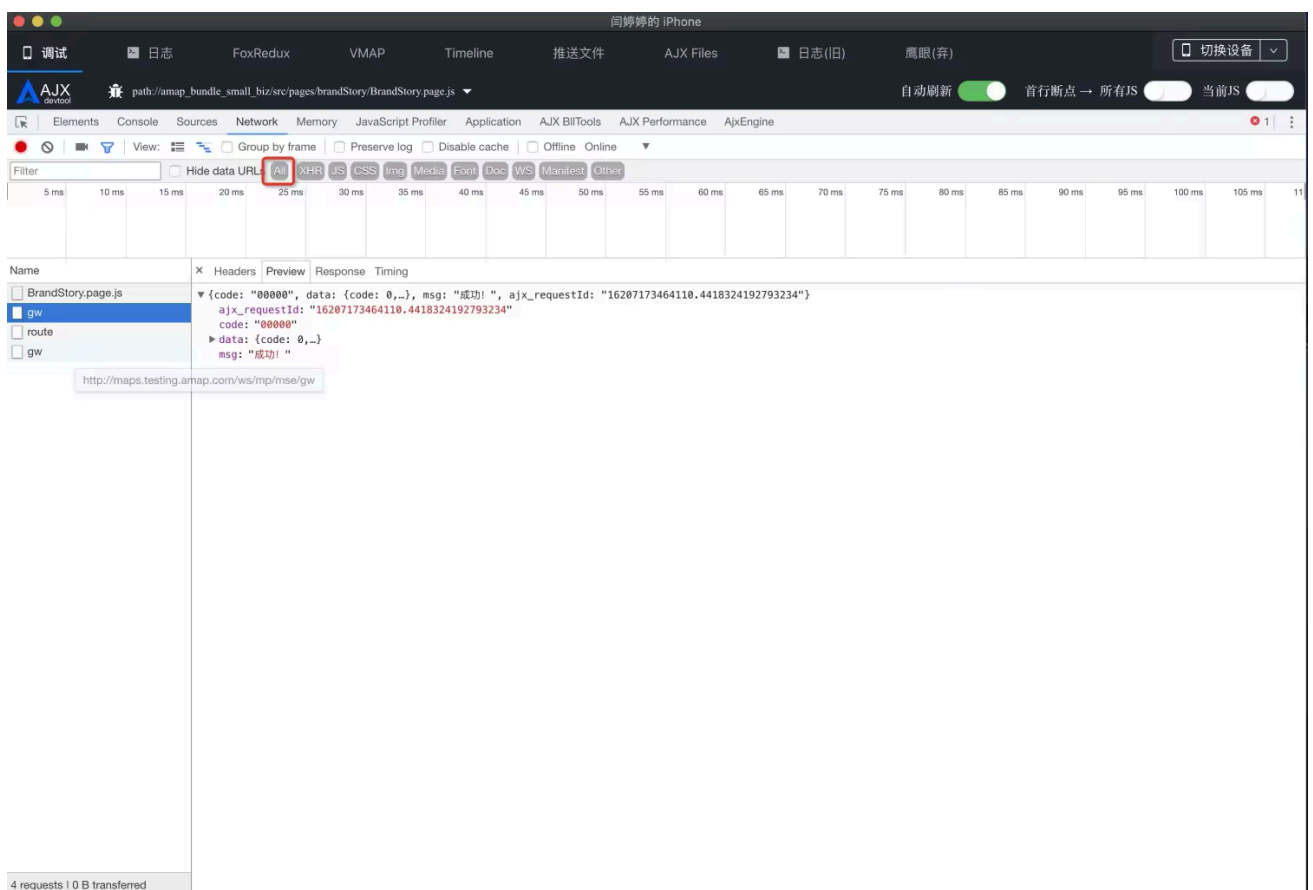
○

网络请求模块之前有一段时间是可以查看到所有请求的接口，但是目前一段时间，突然无法显示了，看着这空空如也的界面，再看看超级简单的文档描述，突然凌乱了，幸好请教了负责工具的同事，很快的解决了。原来也是IDE的一个显示bug，看似网络请求里All已经选中，实际并未选中，需要手动再选中一次。

bug中：

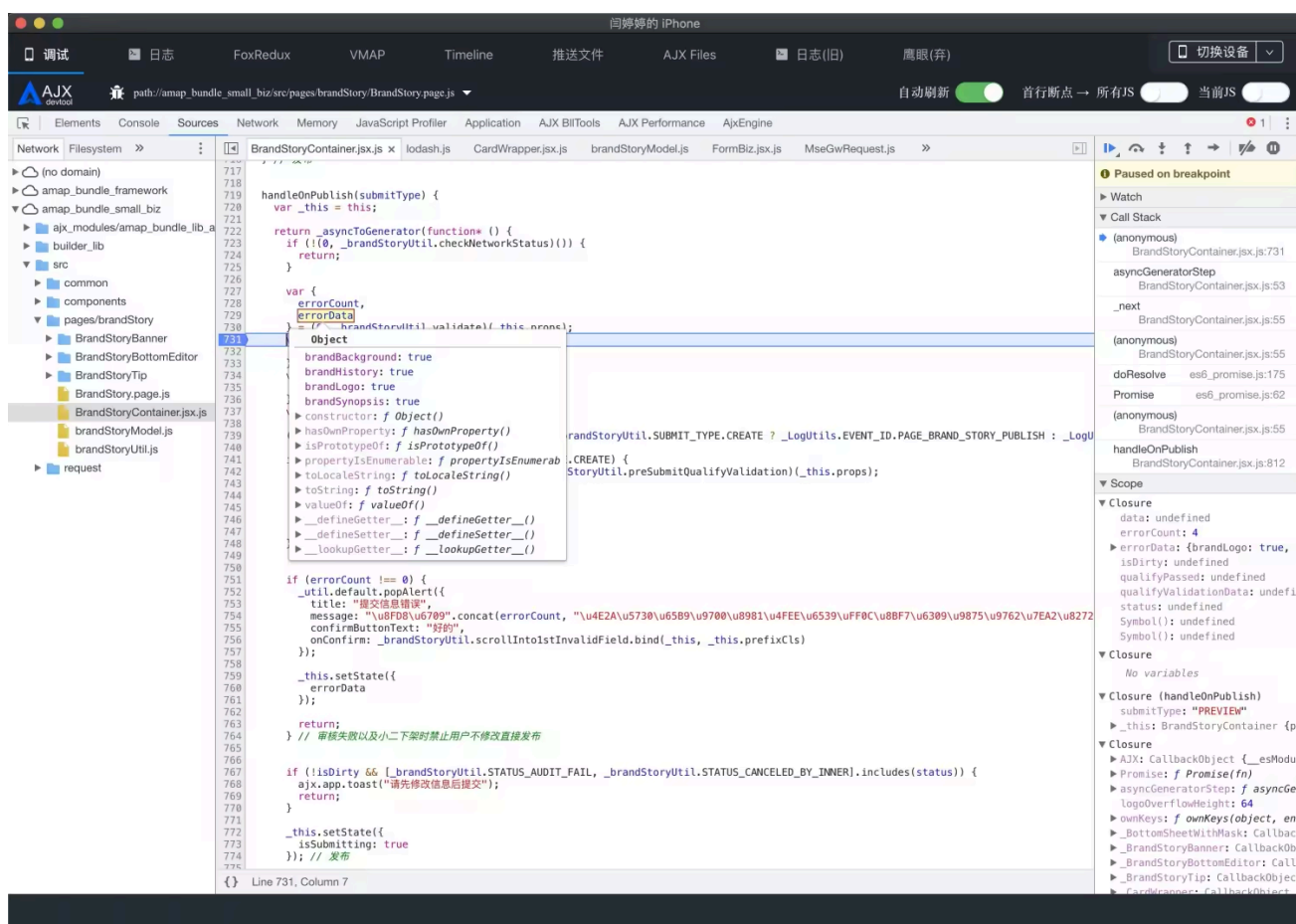


解决后：



断点问题

虽然文档描述，实际看到的界面都和浏览器的断点调试一模一样，但是实际使用就一言难尽了，建议不要依赖此调试，因为时而好使，时而不好使，找过负责工具的同事看过，结果是IED有bug。建议使用print输出log。



展望未来

虽然上面说了很多问题，但是好在AJX是我们内部自己开发的框架，想要迅速找到对应负责人排查解决问题也是相当容易的，所有的负责同事响应都很及时。想想如果使用了RN，可能需要花费大量时间在搜索问题，或者在社区提问，等待回复。

其实在最开始，我也一度认为像Ajs这样的框架是也就是为了KPI而造的一个轮子。但是经过最近对它的研究，看它一路走来的历程，我的想法发生了很大的改变。

不可否认，AJX确实有很多不足之处，但是像React、Vue这样优秀的框架也都少不了吐槽的声音。在阿里，这些技术框架都是伴随着业务痛点出生，并根据业务发展快速迭代，最后如果满足不了业务诉求就会再重新审视当前架构进行调整，这是个非常正常的过程。任何技术的迭代不可能都是非常顺利的，并且还需要具有技术前瞻性来解决之前遇到的问题也是非常困难的。我们应该包容多些，多给它们一些时间，其实我们也能发现，像 Ajs 也从1.0升级到了现在的3.0，一直在不断创新，不断调整，追求更加极致的性能和体验。

贡献

[动态UI引擎 切换JavaScriptCore引擎](#)

[Acorn物料平台](#)

[高德 AJAX bundle 化实践总结](#)

[AMAP跨端技术演变](#)

[高德前端框架思考与展望](#)

[高德前端这五年](#)

[高德共享前端线上稳定性建设历程](#)